

CODEGNAN

FLASK STUDENT HANDBOOK

Student Edition • Python Full Stack Development

Volume	Coverage
Volume I	Flask Foundations — Days 1–5
Volume II	Student Notes Manager — Days 6–20
Volume III	E-Commerce, React Integration & Production Deployment — Days 21–35

Understand the concept. Build the feature. Test it. Debug it. Explain it.

How to Use This Student Handbook

The Codegnan Flask Student Handbook is a learner-focused, project-based resource for the Flask module in Python Full Stack Development. It is structured as one continuous learning book with three detailed volumes. The first volume builds foundations, the second develops the Student Notes Manager, and the third develops and deploys an e-commerce application with React integration.

Student use: Read the concept before class, type every example yourself, complete the guided lab, test both success and failure cases, and write a short reflection after each day. Do not copy code without understanding the request flow.

Recommended Daily Learning Routine

Phase	Suggested share	Student action
Concept and whiteboard	25%	Explain the problem, vocabulary, architecture and request flow.
Guided coding	35%	Build a minimal working path while narrating decisions.
Student lab	30%	Learners reproduce and extend the feature independently.
Review and viva	10%	Test, debug, explain and commit the milestone.

Assessment Standard

- Learners must explain the complete request lifecycle, not only show a working screen.
- Every database operation must be parameterized and accompanied by validation.
- Authentication and authorization are assessed separately.
- Every project milestone must include at least one negative test.
- Git history, README quality and ability to debug are part of the assessment.

Complete 35-Day Curriculum Map

Day	Topic	Volume
1	Introduction to Web Development & Flask	I
2	Environment Setup & First Flask Application	I
3	Routing in Flask	I
4	URL Building & Request Methods	I
5	Request Handling & User Input	I
6	Project Introduction & Database Design	II
7	Project Initialization & UI Design	II
8	Jinja2 Templates & Forms Handling	II
9	Registration Module	II
10	Email Communication & OTP Verification	II
11	Login & Session Management	II
12	Forgot Password System	II
13	Notes CRUD - Part 1	II
14	Notes CRUD - Part 2	II
15	File Uploading & Storage	II
16	File Operations & Logout	II
17	Search, Git & Github Basics	II
18	API Development Basics	II
19	Cookies & Advanced Session Management	II
20	Mini Project Review & Revision	II
21	E-Commerce Project Setup	III
22	Admin Authentication	III
23	Admin Dashboard	III
24	Product Management	III
25	Product Operations	III
26	User Dashboard	III
27	Cart Management	III
28	Razorpay Payment Gateway	III
29	Orders & Invoice Generation	III
30	AWS Account & AWS Deployment	III
31	Final Testing & Production Deployment	III
32	React Integration & Production Build	III
33	AWS EC2 Production Deployment	III
34	Duck DNS & SSL Configuration	III
35	Final Production Testing	III

VOLUME I

FLASK FOUNDATIONS

Days 1–5

DAY 1

Introduction to Web Development & Flask

Student Chapter

Learning outcome: By the end of Day 1, you should be able to explain, implement, test and debug the core outcomes of Introduction to Web Development & Flask.

Day 1 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Introduction to Web Development & Flask.
- Demonstrate the end-to-end request or feature flow for Introduction to Web Development & Flask.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Introduction to Web Development

Introduction to Web Development is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Introduction to Web Development in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. What is Full Stack Development

What is Full Stack Development is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain What is Full Stack Development in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Client-Server Architecture

Client-Server Architecture describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain Client-Server Architecture in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. What is the Internet

What is the Internet is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain What is the Internet in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. What is a Browser

What is a Browser is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain What is a Browser in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. What is a Web Server

What is a Web Server is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain What is a Web Server in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. HTTP & HTTPS

HTTP & HTTPS is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain HTTP & HTTPS in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Request Response Cycle

Request Response Cycle describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain Request Response Cycle in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. What is a Web Application

What is a Web Application is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain What is a Web Application in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. What is Flask

What is Flask is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain What is Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. History of Flask

History of Flask is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain History of Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

12. Features of Flask

Features of Flask is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Features of Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

13. Advantages of Flask

Advantages of Flask is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Advantages of Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

14. Disadvantages of Flask

Disadvantages of Flask is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Disadvantages of Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

15. Applications of Flask

Applications of Flask is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Applications of Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

16. Companies using Flask

Companies using Flask is a core part of Introduction to Web Development & Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Companies using Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Introduction to Web Development & Flask flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 1 reference example

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return '<h1>Welcome to Flask</h1>'

if __name__ == '__main__':
    app.run(debug=True)
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Introduction to Web Development & Flask, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

1. Reproduce the failure consistently.
2. Read the status code and traceback from the top-level error to the relevant application line.
3. Print or log only safe diagnostic values.
4. Verify the active route, method and request payload.
5. Test the database or external service independently.
6. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Build a small Flask exercise focused on Introduction to Web Development & Flask. Include at least three routes or test cases, validation, meaningful responses and a README explaining how to run it.

Minimum Acceptance Criteria

- The Day 1 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 1 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Introduction to Web Development & Flask.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 2

Environment Setup & First Flask Application

Student Chapter

Learning outcome: By the end of Day 2, you should be able to explain, implement, test and debug the core outcomes of Environment Setup & First Flask Application.

Day 2 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Environment Setup & First Flask Application.
- Demonstrate the end-to-end request or feature flow for Environment Setup & First Flask Application.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Python Installation

Python Installation is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Python Installation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. VS Code Setup

VS Code Setup is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain VS Code Setup in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Flask Installation

Flask Installation is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Flask Installation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Creating Virtual Environment

Creating Virtual Environment is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Creating Virtual Environment in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Project Directory Structure

Project Directory Structure is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Project Directory Structure in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. First Flask Application

First Flask Application is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain First Flask Application in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Running Flask Server

Running Flask Server is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Running Flask Server in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Debug Mode

Debug Mode is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Debug Mode in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Flask Auto Reload

Flask Auto Reload is a core part of Environment Setup & First Flask Application. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Flask Auto Reload in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Environment Setup & First Flask Application flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 2 reference example

```
# Create and activate an isolated environment
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
python -m pip install Flask
python -m pip freeze > requirements.txt
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Environment Setup & First Flask Application, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

7. Reproduce the failure consistently.
8. Read the status code and traceback from the top-level error to the relevant application line.
9. Print or log only safe diagnostic values.
10. Verify the active route, method and request payload.
11. Test the database or external service independently.
12. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Build a small Flask exercise focused on Environment Setup & First Flask Application. Include at least three routes or test cases, validation, meaningful responses and a README explaining how to run it.

Minimum Acceptance Criteria

- The Day 2 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 2 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Environment Setup & First Flask Application.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 3

Routing in Flask

Student Chapter

Learning outcome: By the end of Day 3, you should be able to explain, implement, test and debug the core outcomes of Routing in Flask.

Day 3 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Routing in Flask.
- Demonstrate the end-to-end request or feature flow for Routing in Flask.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. URL & ENDPOINT

URL & ENDPOINT is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain URL & ENDPOINT in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. What is Routing

What is Routing is a core part of Routing in Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain What is Routing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Static Routing

Static Routing belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Static Routing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Dynamic Routing

Dynamic Routing is a core part of Routing in Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Dynamic Routing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. URL Parameters

URL Parameters is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain URL Parameters in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Multiple Routes

Multiple Routes is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Multiple Routes in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. URL Converters: string

URL Converters: string is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain URL Converters: string in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. int

int is a core part of Routing in Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain int in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. float

float is a core part of Routing in Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain float in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. path

path is a core part of Routing in Flask. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain path in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. uuid

uuid belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain uuid in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Routing in Flask flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 3 reference example

```
@app.route('/learners/<int:student_id>')
def student_detail(student_id):
    return {'student_id': student_id}

@app.route('/files/<path:filename>')
def file_route(filename):
    return f'Requested: {filename}'
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Routing in Flask, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

13. Reproduce the failure consistently.
14. Read the status code and traceback from the top-level error to the relevant application line.
15. Print or log only safe diagnostic values.
16. Verify the active route, method and request payload.
17. Test the database or external service independently.
18. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Build a small Flask exercise focused on Routing in Flask. Include at least three routes or test cases, validation, meaningful responses and a README explaining how to run it.

Minimum Acceptance Criteria

- The Day 3 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 3 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Routing in Flask.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 4

URL Building & Request Methods

Student Chapter

Learning outcome: By the end of Day 4, you should be able to explain, implement, test and debug the core outcomes of URL Building & Request Methods.

Day 4 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of URL Building & Request Methods.
- Demonstrate the end-to-end request or feature flow for URL Building & Request Methods.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. url_for()

`url_for()` is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain `url_for()` in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. redirect()

`redirect()` is a core part of URL Building & Request Methods. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain `redirect()` in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. HTTP Methods Overview

HTTP Methods Overview is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain HTTP Methods Overview in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. GET Requests

GET Requests is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain GET Requests in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. POST Requests

POST Requests is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain POST Requests in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. PUT Requests

PUT Requests is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain PUT Requests in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. DELETE Requests

DELETE Requests is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain DELETE Requests in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the URL Building & Request Methods flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 4 reference example

```
from flask import redirect, url_for

@app.route('/legacy')
def legacy():
    return redirect(url_for('home'))

@app.route('/resource', methods=['GET', 'POST'])
def resource():
    return {'method': request.method}
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For URL Building & Request Methods, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

19. Reproduce the failure consistently.
20. Read the status code and traceback from the top-level error to the relevant application line.
21. Print or log only safe diagnostic values.
22. Verify the active route, method and request payload.
23. Test the database or external service independently.
24. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Build a small Flask exercise focused on URL Building & Request Methods. Include at least three routes or test cases, validation, meaningful responses and a README explaining how to run it.

Minimum Acceptance Criteria

- The Day 4 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 4 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is URL Building & Request Methods.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 5

Request Handling & User Input

Student Chapter

Learning outcome: By the end of Day 5, you should be able to explain, implement, test and debug the core outcomes of Request Handling & User Input.

Day 5 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Request Handling & User Input.
- Demonstrate the end-to-end request or feature flow for Request Handling & User Input.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Request Object

Request Object is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Request Object in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. request.args

request.args is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain request.args in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. request.form

request.form is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain request.form in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. request.get_json()

`request.get_json()` is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain `request.get_json()` in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. request.method

`request.method` is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain `request.method` in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Handling User Input

Handling User Input is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Handling User Input in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Basic Validation

Basic Validation is a core part of Request Handling & User Input. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Basic Validation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Request Handling & User Input flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 5 reference example

```
from flask import request, jsonify

@app.post('/api/enquiries')
def create_enquiry():
    payload = request.get_json(silent=True) or {}
    name = payload.get('name', '').strip()
    if not name:
        return jsonify(error='name is required'), 400
    return jsonify(message='created', name=name), 201
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Request Handling & User Input, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

25. Reproduce the failure consistently.
26. Read the status code and traceback from the top-level error to the relevant application line.
27. Print or log only safe diagnostic values.
28. Verify the active route, method and request payload.
29. Test the database or external service independently.
30. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Build a small Flask exercise focused on Request Handling & User Input. Include at least three routes or test cases, validation, meaningful responses and a README explaining how to run it.

Minimum Acceptance Criteria

- The Day 5 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 5 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Request Handling & User Input.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

VOLUME II

STUDENT NOTES MANAGER

Days 6–20

DAY 6

Project Introduction & Database Design

Student Chapter

Learning outcome: By the end of Day 6, you should be able to explain, implement, test and debug the core outcomes of Project Introduction & Database Design.

Day 6 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Project Introduction & Database Design.
- Demonstrate the end-to-end request or feature flow for Project Introduction & Database Design.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Introduction to SNM

Introduction to SNM is a core part of Project Introduction & Database Design. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Introduction to SNM in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Software Requirement Analysis

Software Requirement Analysis belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Software Requirement Analysis in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Functional Requirements

Functional Requirements belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Functional Requirements in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Non Functional Requirements

Non Functional Requirements belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Non Functional Requirements in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. User Flow

User Flow describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain User Flow in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Project Workflow

Project Workflow describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain Project Workflow in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Database Planning

Database Planning belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Database Planning in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Entity Relationship

Entity Relationship belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Entity Relationship in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Users Table

Users Table belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Users Table in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. Notes Table

Notes Table belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Notes Table in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. FileData Table

FileData Table belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain FileData Table in your own words using the current project. Then write one possible failure or security risk and how you would test it.

12. Primary Key

Primary Key belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Primary Key in your own words using the current project. Then write one possible failure or security risk and how you would test it.

13. Foreign Key

Foreign Key belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Foreign Key in your own words using the current project. Then write one possible failure or security risk and how you would test it.

14. Constraints

Constraints belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Constraints in your own words using the current project. Then write one possible failure or security risk and how you would test it.

15. Create Database

Create Database belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Create Database in your own words using the current project. Then write one possible failure or security risk and how you would test it.

16. Create Tables

Create Tables belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Create Tables in your own words using the current project. Then write one possible failure or security risk and how you would test it.

17. Insert Sample Data

Insert Sample Data is a core part of Project Introduction & Database Design. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Insert Sample Data in your own words using the current project. Then write one possible failure or security risk and how you would test it.

18. Explain ER Diagram

Explain ER Diagram is a core part of Project Introduction & Database Design. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Explain ER Diagram in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Project Introduction & Database Design flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 6 reference example

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  is_verified BOOLEAN DEFAULT FALSE  
);  
  
CREATE TABLE notes (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL,  
  title VARCHAR(150) NOT NULL,  
  content TEXT NOT NULL,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Project Introduction & Database Design, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

31. Reproduce the failure consistently.
32. Read the status code and traceback from the top-level error to the relevant application line.
33. Print or log only safe diagnostic values.
34. Verify the active route, method and request payload.
35. Test the database or external service independently.
36. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 6 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 6 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 6 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Project Introduction & Database Design.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 7

Project Initialization & UI Design

Student Chapter

Learning outcome: By the end of Day 7, you should be able to explain, implement, test and debug the core outcomes of Project Initialization & UI Design.

Day 7 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Project Initialization & UI Design.
- Demonstrate the end-to-end request or feature flow for Project Initialization & UI Design.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Project Structure (app.py)

Project Structure (app.py) is a core part of Project Initialization & UI Design. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Project Structure (app.py) in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. templates/

templates/ belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain templates/ in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. static/

static/ belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain static/ in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. requirements.txt)

requirements.txt) belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain requirements.txt) in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Why Templates

Why Templates belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Why Templates in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. render_template()

render_template() belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain render_template() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Template Variables

Template Variables belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Template Variables in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Control Statements

Control Statements is a core part of Project Initialization & UI Design. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Control Statements in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Loops

Loops is a core part of Project Initialization & UI Design. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Loops in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. Conditions

Conditions is a core part of Project Initialization & UI Design. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Conditions in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. Filters

Filters is a core part of Project Initialization & UI Design. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Filters in your own words using the current project. Then write one possible failure or security risk and how you would test it.

12. Template Inheritance

Template Inheritance belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Template Inheritance in your own words using the current project. Then write one possible failure or security risk and how you would test it.

13. base.html

base.html belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain base.html in your own words using the current project. Then write one possible failure or security risk and how you would test it.

14. Home Page

Home Page belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Home Page in your own words using the current project. Then write one possible failure or security risk and how you would test it.

15. Register Page

Register Page belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Register Page in your own words using the current project. Then write one possible failure or security risk and how you would test it.

16. Login Page

Login Page belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Login Page in your own words using the current project. Then write one possible failure or security risk and how you would test it.

17. Dashboard UI

Dashboard UI belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Dashboard UI in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Project Initialization & UI Design flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 7 reference example

```
from flask import render_template

@app.get('/')
def home():
    return render_template('home.html', page_title='Student Notes Manager')
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Project Initialization & UI Design, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

37. Reproduce the failure consistently.
38. Read the status code and traceback from the top-level error to the relevant application line.
39. Print or log only safe diagnostic values.
40. Verify the active route, method and request payload.
41. Test the database or external service independently.
42. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 7 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 7 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 7 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Project Initialization & UI Design.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 8

Jinja2 Templates & Forms Handling

Student Chapter

Learning outcome: By the end of Day 8, you should be able to explain, implement, test and debug the core outcomes of Jinja2 Templates & Forms Handling.

Day 8 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Jinja2 Templates & Forms Handling.
- Demonstrate the end-to-end request or feature flow for Jinja2 Templates & Forms Handling.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Jinja2 Template Engine

Jinja2 Template Engine belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Jinja2 Template Engine in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Template Syntax

Template Syntax belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Template Syntax in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Variables in Templates

Variables in Templates belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Variables in Templates in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Control Flow in Templates

Control Flow in Templates describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain Control Flow in Templates in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Loops in Templates

Loops in Templates belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Loops in Templates in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Filters

Filters is a core part of Jinja2 Templates & Forms Handling. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Filters in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Template Inheritance

Template Inheritance belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Template Inheritance in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Block Tags

Block Tags is a core part of Jinja2 Templates & Forms Handling. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Block Tags in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Include Templates

Include Templates belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Include Templates in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. Static Files in Templates

Static Files in Templates belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Static Files in Templates in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Jinja2 Templates & Forms Handling flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 8 reference example

```
{% extends 'base.html' %}
{% block content %}
  {% for note in notes %}
    <article><h2>{{ note.title }}</h2></article>
  {% else %}
    <p>No notes created yet.</p>
  {% endfor %}
{% endblock %}
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Jinja2 Templates & Forms Handling, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

43. Reproduce the failure consistently.
44. Read the status code and traceback from the top-level error to the relevant application line.
45. Print or log only safe diagnostic values.
46. Verify the active route, method and request payload.
47. Test the database or external service independently.
48. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 8 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 8 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 8 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Jinja2 Templates & Forms Handling.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 9

Registration Module

Student Chapter

Learning outcome: By the end of Day 9, you should be able to explain, implement, test and debug the core outcomes of Registration Module.

Day 9 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Registration Module.
- Demonstrate the end-to-end request or feature flow for Registration Module.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. HTML Forms

HTML Forms belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain HTML Forms in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Form GET

Form GET is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Form GET in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Form POST

Form POST is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Form POST in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. request.form

request.form is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain request.form in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Validation

Validation is a core part of Registration Module. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Validation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Introduction to Databases

Introduction to Databases belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Introduction to Databases in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Relational Databases

Relational Databases belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Relational Databases in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. MySQL Basics

MySQL Basics belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain MySQL Basics in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. mysql.connector

mysql.connector belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain mysql.connector in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. Database Connection

Database Connection belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Database Connection in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. Cursor Object

Cursor Object belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Cursor Object in your own words using the current project. Then write one possible failure or security risk and how you would test it.

12. execute()

execute() belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain execute() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

13. commit()

commit() belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain commit() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

14. fetchone()

fetchone() belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain fetchone() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

15. fetchall()

fetchall() belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain fetchall() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

16. Parameterized Queries

Parameterized Queries is a core part of Registration Module. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Parameterized Queries in your own words using the current project. Then write one possible failure or security risk and how you would test it.

17. Registration Form

Registration Form is a core part of Registration Module. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Registration Form in your own words using the current project. Then write one possible failure or security risk and how you would test it.

18. Store User Details

Store User Details is a core part of Registration Module. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Store User Details in your own words using the current project. Then write one possible failure or security risk and how you would test it.

19. Duplicate Email Validation

Duplicate Email Validation is a core part of Registration Module. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Duplicate Email Validation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Registration Module flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 9 reference example

```
sql = 'INSERT INTO users (name,email,password_hash) VALUES (%s,%s,%s) '  
cursor.execute(sql, (name, email, password_hash))  
connection.commit()
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Registration Module, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

49. Reproduce the failure consistently.
50. Read the status code and traceback from the top-level error to the relevant application line.
51. Print or log only safe diagnostic values.
52. Verify the active route, method and request payload.
53. Test the database or external service independently.
54. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 9 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 9 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 9 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Registration Module.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 10

Email Communication & OTP Verification

Student Chapter

Learning outcome: By the end of Day 10, you should be able to explain, implement, test and debug the core outcomes of Email Communication & OTP Verification.

Day 10 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Email Communication & OTP Verification.
- Demonstrate the end-to-end request or feature flow for Email Communication & OTP Verification.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Modularization

Modularization is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Modularization in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. random module

random module is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain random module in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. OTP Generation

OTP Generation is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain OTP Generation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. OTP Verification

OTP Verification is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain OTP Verification in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Account Activation

Account Activation is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Account Activation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Introduction to Email Communication

Introduction to Email Communication is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Introduction to Email Communication in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. SMTP Protocol

SMTP Protocol is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain SMTP Protocol in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. smtplib

smtplib is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain smtplib in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. EmailMessage

EmailMessage is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain EmailMessage in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. Gmail App Password

Gmail App Password is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Gmail App Password in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. Sending Emails using Python

Sending Emails using Python is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Sending Emails using Python in your own words using the current project. Then write one possible failure or security risk and how you would test it.

12. Sending HTML Emails

Sending HTML Emails belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Sending HTML Emails in your own words using the current project. Then write one possible failure or security risk and how you would test it.

13. Error Handling in Email Sending

Error Handling in Email Sending is a core part of Email Communication & OTP Verification. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Error Handling in Email Sending in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Email Communication & OTP Verification flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 10 reference example

```
import secrets
from email.message import EmailMessage

otp = f'{secrets.randbelow(1_000_000):06d}'
message = EmailMessage()
message['Subject'] = 'Verify your account'
message.set_content(f'Your OTP is {otp}')
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Email Communication & OTP Verification, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

55. Reproduce the failure consistently.
56. Read the status code and traceback from the top-level error to the relevant application line.
57. Print or log only safe diagnostic values.
58. Verify the active route, method and request payload.
59. Test the database or external service independently.
60. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 10 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 10 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 10 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Email Communication & OTP Verification.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 11

Login & Session Management

Student Chapter

Learning outcome: By the end of Day 11, you should be able to explain, implement, test and debug the core outcomes of Login & Session Management.

Day 11 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Login & Session Management.
- Demonstrate the end-to-end request or feature flow for Login & Session Management.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Login Validation

Login Validation is a core part of Login & Session Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Login Validation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Password Verification using bcrypt

Password Verification using bcrypt is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Password Verification using bcrypt in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. What is a Session

What is a Session is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain What is a Session in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Types of Sessions

Types of Sessions is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Types of Sessions in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Cookies vs Sessions

Cookies vs Sessions is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Cookies vs Sessions in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Session Configuration in Flask

Session Configuration in Flask is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session Configuration in Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Session Creation

Session Creation is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session Creation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Session Validation (Authentication Check)

Session Validation (Authentication Check) is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session Validation (Authentication Check) in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Protected Routes

Protected Routes is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Protected Routes in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. Session Expiry

Session Expiry is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session Expiry in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. Logout using session.pop()

Logout using session.pop() is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Logout using session.pop() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

12. Login Practical

Login Practical is a core part of Login & Session Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Login Practical in your own words using the current project. Then write one possible failure or security risk and how you would test it.

13. Dashboard Practical

Dashboard Practical belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Dashboard Practical in your own words using the current project. Then write one possible failure or security risk and how you would test it.

14. Logout Practical

Logout Practical is a core part of Login & Session Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Logout Practical in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Login & Session Management flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 11 reference example

```
from flask import session

session.clear()
session['user_id'] = user['id']
session['user_name'] = user['name']

@app.get('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Login & Session Management, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

61. Reproduce the failure consistently.
62. Read the status code and traceback from the top-level error to the relevant application line.
63. Print or log only safe diagnostic values.
64. Verify the active route, method and request payload.
65. Test the database or external service independently.
66. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 11 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 11 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 11 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Login & Session Management.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 12

Forgot Password System

Student Chapter

Learning outcome: By the end of Day 12, you should be able to explain, implement, test and debug the core outcomes of Forgot Password System.

Day 12 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Forgot Password System.
- Demonstrate the end-to-end request or feature flow for Forgot Password System.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Why Tokenization

Why Tokenization is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Why Tokenization in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. itsdangerous

itsdangerous is a core part of Forgot Password System. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain itsdangerous in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. URLSafeTimedSerializer

URLSafeTimedSerializer is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain URLSafeTimedSerializer in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. dumps()

dumps() is a core part of Forgot Password System. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain dumps() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. loads()

loads() is a core part of Forgot Password System. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain loads() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Token Generation

Token Generation is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Token Generation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Token Validation

Token Validation is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Token Validation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Token Expiry

Token Expiry is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Token Expiry in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Reset Password

Reset Password is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Reset Password in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Forgot Password System flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 12 reference example

```
from itsdangerous import URLSafeTimedSerializer, BadSignature, SignatureExpired
serializer = URLSafeTimedSerializer(app.config['SECRET_KEY'])
token = serializer.dumps({'user_id': user_id}, salt='reset-password')
data = serializer.loads(token, salt='reset-password', max_age=900)
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Forgot Password System, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

67. Reproduce the failure consistently.
68. Read the status code and traceback from the top-level error to the relevant application line.
69. Print or log only safe diagnostic values.
70. Verify the active route, method and request payload.
71. Test the database or external service independently.
72. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 12 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 12 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 12 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Forgot Password System.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 13

Notes CRUD - Part 1

Student Chapter

Learning outcome: By the end of Day 13, you should be able to explain, implement, test and debug the core outcomes of Notes CRUD - Part 1.

Day 13 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Notes CRUD - Part 1.
- Demonstrate the end-to-end request or feature flow for Notes CRUD - Part 1.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Add Notes

Add Notes is a core part of Notes CRUD - Part 1. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Add Notes in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. View All Notes

View All Notes is a core part of Notes CRUD - Part 1. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain View All Notes in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Session Validation

Session Validation is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session Validation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Database Insertion

Database Insertion belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Database Insertion in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Notes CRUD - Part 1 flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 13 reference example

```
@app.post('/notes')
def add_note():
    require_login()
    title = request.form.get('title','').strip()
    content = request.form.get('content','').strip()
    cursor.execute('INSERT INTO notes(user_id,title,content) VALUES(%s,%s,%s)',
                   (session['user_id'], title, content))
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Notes CRUD - Part 1, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

73. Reproduce the failure consistently.
74. Read the status code and traceback from the top-level error to the relevant application line.
75. Print or log only safe diagnostic values.
76. Verify the active route, method and request payload.
77. Test the database or external service independently.
78. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 13 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 13 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 13 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Notes CRUD - Part 1.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 14

Notes CRUD - Part 2

Student Chapter

Learning outcome: By the end of Day 14, you should be able to explain, implement, test and debug the core outcomes of Notes CRUD - Part 2.

Day 14 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Notes CRUD - Part 2.
- Demonstrate the end-to-end request or feature flow for Notes CRUD - Part 2.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. View Single Note

View Single Note is a core part of Notes CRUD - Part 2. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain View Single Note in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Update Note

Update Note is a core part of Notes CRUD - Part 2. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Update Note in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Delete Note

Delete Note is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Delete Note in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Authorization Check

Authorization Check is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Authorization Check in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Notes CRUD - Part 2 flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 14 reference example

```
cursor.execute('SELECT * FROM notes WHERE id=%s AND user_id=%s',
               (note_id, session['user_id']))
note = cursor.fetchone()
if note is None:
    abort(404)
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Notes CRUD - Part 2, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

79. Reproduce the failure consistently.
80. Read the status code and traceback from the top-level error to the relevant application line.
81. Print or log only safe diagnostic values.
82. Verify the active route, method and request payload.
83. Test the database or external service independently.
84. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 14 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 14 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 14 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Notes CRUD - Part 2.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 15

File Uploading & Storage

Student Chapter

Learning outcome: By the end of Day 15, you should be able to explain, implement, test and debug the core outcomes of File Uploading & Storage.

Day 15 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of File Uploading & Storage.
- Demonstrate the end-to-end request or feature flow for File Uploading & Storage.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Upload File

Upload File involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Upload File in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. request.files

request.files is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain request.files in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. secure_filename()

secure_filename() is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain secure_filename() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. File Validation

File Validation involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain File Validation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Duplicate File Check

Duplicate File Check involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Duplicate File Check in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. File Reading & Storage

File Reading & Storage involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain File Reading & Storage in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Store File Metadata in Database

Store File Metadata in Database belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Store File Metadata in Database in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the File Uploading & Storage flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 15 reference example

```
from werkzeug.utils import secure_filename
file = request.files.get('file')
filename = secure_filename(file.filename)
if not allowed_file(filename):
    abort(400, 'Unsupported file type')
file.save(upload_path / filename)
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For File Uploading & Storage, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

85. Reproduce the failure consistently.
86. Read the status code and traceback from the top-level error to the relevant application line.
87. Print or log only safe diagnostic values.
88. Verify the active route, method and request payload.
89. Test the database or external service independently.
90. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 15 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 15 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 15 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is File Uploading & Storage.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 16

File Operations & Logout

Student Chapter

Learning outcome: By the end of Day 16, you should be able to explain, implement, test and debug the core outcomes of File Operations & Logout.

Day 16 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of File Operations & Logout.
- Demonstrate the end-to-end request or feature flow for File Operations & Logout.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. View Uploaded Files

View Uploaded Files involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain View Uploaded Files in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Fetch File Metadata

Fetch File Metadata belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Fetch File Metadata in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Display File List

Display File List involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Display File List in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. View File in Browser

View File in Browser involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain View File in Browser in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Download File

Download File involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Download File in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. send_file()

send_file() involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain send_file() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. BytesIO

BytesIO is a core part of File Operations & Logout. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain BytesIO in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Delete File

Delete File is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Delete File in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Logout using session.pop()

Logout using session.pop() is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Logout using session.pop() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the File Operations & Logout flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 16 reference example

```
from flask import send_file
from io import BytesIO

return send_file(BytesIO(row['content']),
                  download_name=row['original_name'],
                  as_attachment=True)
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For File Operations & Logout, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

91. Reproduce the failure consistently.
92. Read the status code and traceback from the top-level error to the relevant application line.
93. Print or log only safe diagnostic values.
94. Verify the active route, method and request payload.
95. Test the database or external service independently.
96. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 16 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 16 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 16 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is File Operations & Logout.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 17

Search, Git & Github Basics

Student Chapter

Learning outcome: By the end of Day 17, you should be able to explain, implement, test and debug the core outcomes of Search, Git & Github Basics.

Day 17 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Search, Git & Github Basics.
- Demonstrate the end-to-end request or feature flow for Search, Git & Github Basics.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Search Route

Search Route is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Search Route in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Handling POST Request

Handling POST Request is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Handling POST Request in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Input Validation using Regex

Input Validation using Regex is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Input Validation using Regex in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. SQL LIKE Operator

SQL LIKE Operator belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain SQL LIKE Operator in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Display Search Results

Display Search Results is a core part of Search, Git & Github Basics. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Display Search Results in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. flask_excel

flask_excel is a core part of Search, Git & Github Basics. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain flask_excel in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Git Basics

Git Basics supports collaboration, traceability and safe experimentation. Treat each working feature as a small milestone that deserves a clear commit.

Teach status, add, commit, log, branch and push through the current project. A commit message should describe the change and intent, not merely say “updated code”.

Ensure .env, secrets, virtual environments, generated files and local uploads are excluded. Show how history helps identify when a regression was introduced.

Check your understanding: Explain Git Basics in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. GitHub Push

GitHub Push supports collaboration, traceability and safe experimentation. Treat each working feature as a small milestone that deserves a clear commit.

Teach status, add, commit, log, branch and push through the current project. A commit message should describe the change and intent, not merely say “updated code”.

Ensure .env, secrets, virtual environments, generated files and local uploads are excluded. Show how history helps identify when a regression was introduced.

Check your understanding: Explain GitHub Push in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response

```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Search, Git & Github Basics flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 17 reference example

```
term = request.form.get('q','').strip()
pattern = f'%{term}%'
cursor.execute('SELECT * FROM notes WHERE user_id=%s AND title LIKE %s',
               (session['user_id'], pattern))
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Search, Git & Github Basics, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

97. Reproduce the failure consistently.
98. Read the status code and traceback from the top-level error to the relevant application line.
99. Print or log only safe diagnostic values.
100. Verify the active route, method and request payload.
101. Test the database or external service independently.
102. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 17 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 17 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 17 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Search, Git & Github Basics.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 18

API Development Basics

Student Chapter

Learning outcome: By the end of Day 18, you should be able to explain, implement, test and debug the core outcomes of API Development Basics.

Day 18 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of API Development Basics.
- Demonstrate the end-to-end request or feature flow for API Development Basics.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. What is an API

What is an API is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain What is an API in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. REST Architecture

REST Architecture describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain REST Architecture in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. JSON Response

JSON Response is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain JSON Response in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. jsonify()

jsonify() is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain jsonify() in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. API Endpoints

API Endpoints is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain API Endpoints in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Status Codes

Status Codes is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Status Codes in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. API Testing using Postman

API Testing using Postman is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain API Testing using Postman in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the API Development Basics flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 18 reference example

```
@app.get('/api/notes/<int:note_id>')
def api_note(note_id):
    note = find_owned_note(note_id, session['user_id'])
    if not note:
        return jsonify(error='not found'), 404
    return jsonify(note), 200
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For API Development Basics, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

103. Reproduce the failure consistently.
104. Read the status code and traceback from the top-level error to the relevant application line.
105. Print or log only safe diagnostic values.
106. Verify the active route, method and request payload.
107. Test the database or external service independently.
108. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 18 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 18 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 18 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is API Development Basics.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 19

Cookies & Advanced Session Management

Student Chapter

Learning outcome: By the end of Day 19, you should be able to explain, implement, test and debug the core outcomes of Cookies & Advanced Session Management.

Day 19 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Cookies & Advanced Session Management.
- Demonstrate the end-to-end request or feature flow for Cookies & Advanced Session Management.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. What are Cookies

What are Cookies is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain What are Cookies in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Setting Cookies

Setting Cookies is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Setting Cookies in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Reading Cookies

Reading Cookies is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Reading Cookies in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Deleting Cookies

Deleting Cookies is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Deleting Cookies in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Cookie Expiry

Cookie Expiry is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Cookie Expiry in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Secure Cookies

Secure Cookies is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Secure Cookies in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Session vs Cookies Use Cases

Session vs Cookies Use Cases is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session vs Cookies Use Cases in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Cookies & Advanced Session Management flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 19 reference example

```
response = make_response(render_template('preferences.html'))
response.set_cookie('theme', 'dark', max_age=60*60*24*30,
                    httponly=True, secure=True, samesite='Lax')
return response
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Cookies & Advanced Session Management, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

109. Reproduce the failure consistently.
110. Read the status code and traceback from the top-level error to the relevant application line.
111. Print or log only safe diagnostic values.
112. Verify the active route, method and request payload.
113. Test the database or external service independently.
114. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 19 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 19 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 19 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Cookies & Advanced Session Management.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 20

Mini Project Review & Revision

Student Chapter

Learning outcome: By the end of Day 20, you should be able to explain, implement, test and debug the core outcomes of Mini Project Review & Revision.

Day 20 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Mini Project Review & Revision.
- Demonstrate the end-to-end request or feature flow for Mini Project Review & Revision.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. SNM Project Complete Review

SNM Project Complete Review is a core part of Mini Project Review & Revision. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain SNM Project Complete Review in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Code Walkthrough

Code Walkthrough is a core part of Mini Project Review & Revision. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Code Walkthrough in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Bug Fixing

Bug Fixing is a core part of Mini Project Review & Revision. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Bug Fixing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Best Practices Discussion

Best Practices Discussion is a core part of Mini Project Review & Revision. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Best Practices Discussion in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Q&A Session

Q&A Session is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Q&A Session in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Mini Project Review & Revision flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 20 reference example

```
# Final review commands
python -m compileall .
python -m pip freeze > requirements.txt
git status
git add .
git commit -m 'Complete Student Notes Manager milestone'
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Mini Project Review & Revision, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

115. Reproduce the failure consistently.
116. Read the status code and traceback from the top-level error to the relevant application line.
117. Print or log only safe diagnostic values.
118. Verify the active route, method and request payload.
119. Test the database or external service independently.
120. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 20 milestone to the Student Notes Manager. Commit only after the feature passes the acceptance checklist and another student has tested one failure path.

Minimum Acceptance Criteria

- The Day 20 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 20 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Mini Project Review & Revision.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

VOLUME III

E-COMMERCE, REACT & DEPLOYMENT

Days 21–35

DAY 21

E-Commerce Project Setup

Student Chapter

Learning outcome: By the end of Day 21, you should be able to explain, implement, test and debug the core outcomes of E-Commerce Project Setup.

Day 21 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of E-Commerce Project Setup.
- Demonstrate the end-to-end request or feature flow for E-Commerce Project Setup.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Project Introduction

Project Introduction is a core part of E-Commerce Project Setup. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Project Introduction in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Project Structure

Project Structure is a core part of E-Commerce Project Setup. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Project Structure in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Database Design

Database Design belongs to the persistence layer. It determines how application data is represented, validated, connected and retrieved after the current request has finished.

Teach the database contract before showing Python code: column names, data types, nullability, uniqueness, relationships and indexes. Then map each form field or API property to the correct column.

Use parameterized SQL for user-controlled values. Explain transaction boundaries, when commit is required and why database errors must not expose credentials or raw internal details to the user.

Check your understanding: Explain Database Design in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Admin Flow

Admin Flow describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain Admin Flow in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. User Flow

User Flow describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain User Flow in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response

```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the E-Commerce Project Setup flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 21 reference example

```
ecommerce/  
├── app.py  
├── config.py  
├── services/  
├── templates/  
├── static/  
├── uploads/  
└── requirements.txt
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For E-Commerce Project Setup, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

121. Reproduce the failure consistently.
122. Read the status code and traceback from the top-level error to the relevant application line.
123. Print or log only safe diagnostic values.
124. Verify the active route, method and request payload.
125. Test the database or external service independently.
126. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 21 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 21 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 21 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is E-Commerce Project Setup.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 22

Admin Authentication

Student Chapter

Learning outcome: By the end of Day 22, you should be able to explain, implement, test and debug the core outcomes of Admin Authentication.

Day 22 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Admin Authentication.
- Demonstrate the end-to-end request or feature flow for Admin Authentication.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Admin Registration

Admin Registration is a core part of Admin Authentication. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Admin Registration in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. OTP Verification

OTP Verification is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain OTP Verification in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Password Hashing using bcrypt

Password Hashing using bcrypt is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Password Hashing using bcrypt in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Admin Login

Admin Login is a core part of Admin Authentication. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Admin Login in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Session Management

Session Management is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session Management in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. API Testing using Postman

API Testing using Postman is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain API Testing using Postman in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response

```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Admin Authentication flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 22 reference example

```
password_hash = bcrypt.generate_password_hash(password).decode('utf-8')
if bcrypt.check_password_hash(admin['password_hash'], password):
    session.clear(); session['admin_id'] = admin['id']
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Admin Authentication, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

127. Reproduce the failure consistently.
128. Read the status code and traceback from the top-level error to the relevant application line.
129. Print or log only safe diagnostic values.
130. Verify the active route, method and request payload.
131. Test the database or external service independently.
132. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 22 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 22 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 22 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Admin Authentication.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 23

Admin Dashboard

Student Chapter

Learning outcome: By the end of Day 23, you should be able to explain, implement, test and debug the core outcomes of Admin Dashboard.

Day 23 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Admin Dashboard.
- Demonstrate the end-to-end request or feature flow for Admin Dashboard.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Dashboard Access Control

Dashboard Access Control belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Dashboard Access Control in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Admin Profile Update

Admin Profile Update involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Admin Profile Update in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Profile Image Upload

Profile Image Upload involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Profile Image Upload in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Admin Dashboard flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 23 reference example

```
def admin_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if 'admin_id' not in session:
            return redirect(url_for('admin_login'))
        return view(*args, **kwargs)
    return wrapped
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Admin Dashboard, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

133. Reproduce the failure consistently.
134. Read the status code and traceback from the top-level error to the relevant application line.
135. Print or log only safe diagnostic values.
136. Verify the active route, method and request payload.
137. Test the database or external service independently.
138. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 23 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 23 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 23 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Admin Dashboard.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 24

Product Management

Student Chapter

Learning outcome: By the end of Day 24, you should be able to explain, implement, test and debug the core outcomes of Product Management.

Day 24 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Product Management.
- Demonstrate the end-to-end request or feature flow for Product Management.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Add Product

Add Product is a core part of Product Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Add Product in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Image Upload

Image Upload involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Image Upload in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Store Images in static

Store Images in static belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Store Images in static in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Product Listing

Product Listing is a core part of Product Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Product Listing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Single Product View

Single Product View is a core part of Product Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Single Product View in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Product Management flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 24 reference example

```
INSERT_PRODUCT = '''
INSERT INTO products(name,description,price,category_id,image_name)
VALUES (%s,%s,%s,%s,%s)
'''
cursor.execute(INSERT_PRODUCT, (name,description,price,category_id,image_name))
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Product Management, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

139. Reproduce the failure consistently.
140. Read the status code and traceback from the top-level error to the relevant application line.
141. Print or log only safe diagnostic values.
142. Verify the active route, method and request payload.
143. Test the database or external service independently.
144. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 24 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 24 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 24 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Product Management.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 25

Product Operations

Student Chapter

Learning outcome: By the end of Day 25, you should be able to explain, implement, test and debug the core outcomes of Product Operations.

Day 25 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Product Operations.
- Demonstrate the end-to-end request or feature flow for Product Operations.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Update Product

Update Product is a core part of Product Operations. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Update Product in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Replace Image

Replace Image involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Replace Image in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Delete Product

Delete Product is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Delete Product in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Search Filters

Search Filters is a core part of Product Operations. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Search Filters in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Category Filters

Category Filters is a core part of Product Operations. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Category Filters in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Product Operations flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 25 reference example

```
UPDATE products
SET name=%s, description=%s, price=%s, category_id=%s, image_name=%s
WHERE id=%s
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Product Operations, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

- 145.Reproduce the failure consistently.
- 146.Read the status code and traceback from the top-level error to the relevant application line.
- 147.Print or log only safe diagnostic values.
- 148.Verify the active route, method and request payload.
- 149.Test the database or external service independently.
- 150.Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 25 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 25 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 25 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Product Operations.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 26

User Dashboard

Student Chapter

Learning outcome: By the end of Day 26, you should be able to explain, implement, test and debug the core outcomes of User Dashboard.

Day 26 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of User Dashboard.
- Demonstrate the end-to-end request or feature flow for User Dashboard.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Product Listing

Product Listing is a core part of User Dashboard. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Product Listing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Product Description

Product Description is a core part of User Dashboard. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Product Description in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Search Functionality

Search Functionality is a core part of User Dashboard. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Search Functionality in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Category Filtering

Category Filtering is a core part of User Dashboard. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Category Filtering in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the User Dashboard flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 26 reference example

```
query = 'SELECT * FROM products WHERE is_active=TRUE'
params=[]
if category_id:
    query += ' AND category_id=%s'; params.append(category_id)
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For User Dashboard, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

151. Reproduce the failure consistently.
152. Read the status code and traceback from the top-level error to the relevant application line.
153. Print or log only safe diagnostic values.
154. Verify the active route, method and request payload.
155. Test the database or external service independently.
156. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 26 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 26 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 26 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is User Dashboard.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 27

Cart Management

Student Chapter

Learning outcome: By the end of Day 27, you should be able to explain, implement, test and debug the core outcomes of Cart Management.

Day 27 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Cart Management.
- Demonstrate the end-to-end request or feature flow for Cart Management.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Add to Cart

Add to Cart is a core part of Cart Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Add to Cart in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. View Cart

View Cart is a core part of Cart Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain View Cart in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Remove Item

Remove Item is a core part of Cart Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Remove Item in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Quantity Increase

Quantity Increase is a core part of Cart Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Quantity Increase in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Quantity Decrease

Quantity Decrease is a core part of Cart Management. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Quantity Decrease in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Session-based Cart

Session-based Cart is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session-based Cart in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Cart Management flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 27 reference example

```
cart = session.get('cart', {})  
key = str(product_id)  
cart[key] = cart.get(key, 0) + 1  
session['cart'] = cart  
session.modified = True
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Cart Management, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

157. Reproduce the failure consistently.
158. Read the status code and traceback from the top-level error to the relevant application line.
159. Print or log only safe diagnostic values.
160. Verify the active route, method and request payload.
161. Test the database or external service independently.
162. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 27 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 27 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 27 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Cart Management.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 28

Razorpay Payment Gateway

Student Chapter

Learning outcome: By the end of Day 28, you should be able to explain, implement, test and debug the core outcomes of Razorpay Payment Gateway.

Day 28 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Razorpay Payment Gateway.
- Demonstrate the end-to-end request or feature flow for Razorpay Payment Gateway.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Order Creation

Order Creation is a transactional feature. The browser can initiate a payment, but the server must remain the authority for amount, order identity and final verification.

Create the order on the server, send only necessary checkout data to the client, verify the payment signature on the server and update the order atomically. Never trust a client-only success callback.

Demonstrate duplicate callbacks, failed payments and network interruption. Learners should understand idempotency and why payment records need auditable status transitions.

Check your understanding: Explain Order Creation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. checkout.js Integration

checkout.js Integration is a transactional feature. The browser can initiate a payment, but the server must remain the authority for amount, order identity and final verification.

Create the order on the server, send only necessary checkout data to the client, verify the payment signature on the server and update the order atomically. Never trust a client-only success callback.

Demonstrate duplicate callbacks, failed payments and network interruption. Learners should understand idempotency and why payment records need auditable status transitions.

Check your understanding: Explain checkout.js Integration in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Payment Verification

Payment Verification is a transactional feature. The browser can initiate a payment, but the server must remain the authority for amount, order identity and final verification.

Create the order on the server, send only necessary checkout data to the client, verify the payment signature on the server and update the order atomically. Never trust a client-only success callback.

Demonstrate duplicate callbacks, failed payments and network interruption. Learners should understand idempotency and why payment records need auditable status transitions.

Check your understanding: Explain Payment Verification in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Razorpay Payment Gateway flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 28 reference example

```
import razorpay
client = razorpay.Client(auth=(KEY_ID, KEY_SECRET))
order = client.order.create({'amount': amount_paise, 'currency': 'INR', 'receipt': receipt})
client.utility.verify_payment_signature(signature_payload)
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Razorpay Payment Gateway, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

163. Reproduce the failure consistently.
164. Read the status code and traceback from the top-level error to the relevant application line.
165. Print or log only safe diagnostic values.
166. Verify the active route, method and request payload.
167. Test the database or external service independently.
168. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 28 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 28 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 28 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Razorpay Payment Gateway.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 29

Orders & Invoice Generation

Student Chapter

Learning outcome: By the end of Day 29, you should be able to explain, implement, test and debug the core outcomes of Orders & Invoice Generation.

Day 29 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Orders & Invoice Generation.
- Demonstrate the end-to-end request or feature flow for Orders & Invoice Generation.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Store Orders

Store Orders is a transactional feature. The browser can initiate a payment, but the server must remain the authority for amount, order identity and final verification.

Create the order on the server, send only necessary checkout data to the client, verify the payment signature on the server and update the order atomically. Never trust a client-only success callback.

Demonstrate duplicate callbacks, failed payments and network interruption. Learners should understand idempotency and why payment records need auditable status transitions.

Check your understanding: Explain Store Orders in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Order History

Order History is a transactional feature. The browser can initiate a payment, but the server must remain the authority for amount, order identity and final verification.

Create the order on the server, send only necessary checkout data to the client, verify the payment signature on the server and update the order atomically. Never trust a client-only success callback.

Demonstrate duplicate callbacks, failed payments and network interruption. Learners should understand idempotency and why payment records need auditable status transitions.

Check your understanding: Explain Order History in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. HTML to PDF Invoice

HTML to PDF Invoice belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain HTML to PDF Invoice in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Download Invoice

Download Invoice involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Download Invoice in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Orders & Invoice Generation flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 29 reference example

```
from weasyprint import HTML
html = render_template('invoice.html', order=order, items=items)
pdf_bytes = HTML(string=html).write_pdf()
return send_file(BytesIO(pdf_bytes), mimetype='application/pdf',
                  download_name=f'invoice-{order["id"]}.pdf')
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Orders & Invoice Generation, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

169. Reproduce the failure consistently.
170. Read the status code and traceback from the top-level error to the relevant application line.
171. Print or log only safe diagnostic values.
172. Verify the active route, method and request payload.
173. Test the database or external service independently.
174. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 29 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 29 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 29 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Orders & Invoice Generation.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 30

AWS Account & AWS Deployment

Student Chapter

Learning outcome: By the end of Day 30, you should be able to explain, implement, test and debug the core outcomes of AWS Account & AWS Deployment.

Day 30 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of AWS Account & AWS Deployment.
- Demonstrate the end-to-end request or feature flow for AWS Account & AWS Deployment.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. AWS Account Creation

AWS Account Creation moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain AWS Account Creation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. EC2 Setup

EC2 Setup moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain EC2 Setup in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Gunicorn

Gunicorn moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Gunicorn in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Nginx

Nginx moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Nginx in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Reverse Proxy

Reverse Proxy is a core part of AWS Account & AWS Deployment. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Reverse Proxy in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Environment Variables

Environment Variables moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Environment Variables in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Deploy Flask Application on AWS

Deploy Flask Application on AWS moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Deploy Flask Application on AWS in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
  |
  v
Flask URL rule and HTTP method
  |
  v
Input extraction and validation
  |
  +--> Authentication / authorization
  +--> Database / file / email / payment service
  |
  v
HTML, redirect, file or JSON response

```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the AWS Account & AWS Deployment flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 30 reference example

```
# Gunicorn and Nginx flow
gunicorn --workers 3 --bind 127.0.0.1:8000 'app:create_app()'
# Nginx proxies public requests to 127.0.0.1:8000
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For AWS Account & AWS Deployment, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

175. Reproduce the failure consistently.
176. Read the status code and traceback from the top-level error to the relevant application line.
177. Print or log only safe diagnostic values.
178. Verify the active route, method and request payload.
179. Test the database or external service independently.
180. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 30 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 30 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 30 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is AWS Account & AWS Deployment.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 31

Final Testing & Production Deployment

Student Chapter

Learning outcome: By the end of Day 31, you should be able to explain, implement, test and debug the core outcomes of Final Testing & Production Deployment.

Day 31 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Final Testing & Production Deployment.
- Demonstrate the end-to-end request or feature flow for Final Testing & Production Deployment.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. 404 Error Handling

404 Error Handling is a core part of Final Testing & Production Deployment. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain 404 Error Handling in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. 500 Error Handling

500 Error Handling is a core part of Final Testing & Production Deployment. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain 500 Error Handling in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Logging Basics

Logging Basics moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Logging Basics in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Security Best Practices

Security Best Practices is a core part of Final Testing & Production Deployment. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Security Best Practices in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. User Flow Testing

User Flow Testing describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain User Flow Testing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Admin Flow Testing

Admin Flow Testing describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain Admin Flow Testing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Payment Testing

Payment Testing is a transactional feature. The browser can initiate a payment, but the server must remain the authority for amount, order identity and final verification.

Create the order on the server, send only necessary checkout data to the client, verify the payment signature on the server and update the order atomically. Never trust a client-only success callback.

Demonstrate duplicate callbacks, failed payments and network interruption. Learners should understand idempotency and why payment records need auditable status transitions.

Check your understanding: Explain Payment Testing in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Deployment Demonstration

Deployment Demonstration moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Deployment Demonstration in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Interview Questions & Viva

Interview Questions & Viva is a core part of Final Testing & Production Deployment. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Interview Questions & Viva in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Final Testing & Production Deployment flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 31 reference example

```
@app.errorhandler(404)
def not_found(error):
    return render_template('errors/404.html'), 404

@app.errorhandler(500)
def server_error(error):
    app.logger.exception('Unhandled error')
    return render_template('errors/500.html'), 500
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Final Testing & Production Deployment, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

181. Reproduce the failure consistently.
182. Read the status code and traceback from the top-level error to the relevant application line.
183. Print or log only safe diagnostic values.
184. Verify the active route, method and request payload.
185. Test the database or external service independently.
186. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 31 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 31 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 31 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Final Testing & Production Deployment.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 32

React Integration & Production Build

Student Chapter

Learning outcome: By the end of Day 32, you should be able to explain, implement, test and debug the core outcomes of React Integration & Production Build.

Day 32 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of React Integration & Production Build.
- Demonstrate the end-to-end request or feature flow for React Integration & Production Build.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. React Project Structure

React Project Structure belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain React Project Structure in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. React + Flask Architecture

React + Flask Architecture describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain React + Flask Architecture in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. API Communication Flow

API Communication Flow describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain API Communication Flow in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Axios Setup

Axios Setup belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Axios Setup in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Environment Variables (.env)

Environment Variables (.env) moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Environment Variables (.env) in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. CORS Configuration

CORS Configuration is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain CORS Configuration in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Authentication Flow

Authentication Flow describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain Authentication Flow in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Session-Based Login

Session-Based Login is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Session-Based Login in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Connect Login API

Connect Login API is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Connect Login API in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. Connect Product Listing API

Connect Product Listing API is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Connect Product Listing API in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. Connect Add-To-Cart API

Connect Add-To-Cart API is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Connect Add-To-Cart API in your own words using the current project. Then write one possible failure or security risk and how you would test it.

12. Build React Project

Build React Project belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Build React Project in your own words using the current project. Then write one possible failure or security risk and how you would test it.

13. Generate Production Build Folder

Generate Production Build Folder belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Generate Production Build Folder in your own words using the current project. Then write one possible failure or security risk and how you would test it.

14. Serve React Build with Flask

Serve React Build with Flask belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Serve React Build with Flask in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the React Integration & Production Build flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 32 reference example

```
import axios from 'axios';
const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  withCredentials: true
});
export default api;
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For React Integration & Production Build, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

187. Reproduce the failure consistently.
188. Read the status code and traceback from the top-level error to the relevant application line.
189. Print or log only safe diagnostic values.
190. Verify the active route, method and request payload.
191. Test the database or external service independently.
192. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 32 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 32 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 32 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is React Integration & Production Build.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 33

AWS EC2 Production Deployment

Student Chapter

Learning outcome: By the end of Day 33, you should be able to explain, implement, test and debug the core outcomes of AWS EC2 Production Deployment.

Day 33 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of AWS EC2 Production Deployment.
- Demonstrate the end-to-end request or feature flow for AWS EC2 Production Deployment.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. AWS EC2 Instance Creation

AWS EC2 Instance Creation moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain AWS EC2 Instance Creation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Security Groups

Security Groups is a core part of AWS EC2 Production Deployment. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Security Groups in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Key Pair Configuration

Key Pair Configuration is a core part of AWS EC2 Production Deployment. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Key Pair Configuration in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Linux Commands Required for Deployment

Linux Commands Required for Deployment belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Linux Commands Required for Deployment in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Python Environment Setup

Python Environment Setup is a core part of AWS EC2 Production Deployment. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Python Environment Setup in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Upload Project to Server

Upload Project to Server involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Upload Project to Server in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Install Requirements

Install Requirements belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Install Requirements in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Configure Gunicorn

Configure Gunicorn moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Configure Gunicorn in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Create Systemd Service

Create Systemd Service moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Create Systemd Service in your own words using the current project. Then write one possible failure or security risk and how you would test it.

10. Configure Nginx Reverse Proxy

Configure Nginx Reverse Proxy moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Configure Nginx Reverse Proxy in your own words using the current project. Then write one possible failure or security risk and how you would test it.

11. Serve Flask API through Nginx

Serve Flask API through Nginx is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Serve Flask API through Nginx in your own words using the current project. Then write one possible failure or security risk and how you would test it.

12. Deploy React Build

Deploy React Build belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Deploy React Build in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the AWS EC2 Production Deployment flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 33 reference example

```
[Unit]
Description=Codegnan Flask Application
After=network.target

[Service]
WorkingDirectory=/srv/codegnan-app
EnvironmentFile=/srv/codegnan-app/.env
ExecStart=/srv/codegnan-app/.venv/bin/gunicorn -w 3 -b 127.0.0.1:8000 'app:create_app()'
Restart=always
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For AWS EC2 Production Deployment, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

193. Reproduce the failure consistently.
194. Read the status code and traceback from the top-level error to the relevant application line.
195. Print or log only safe diagnostic values.
196. Verify the active route, method and request payload.
197. Test the database or external service independently.
198. Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 33 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 33 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 33 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is AWS EC2 Production Deployment.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 34

Duck DNS & SSL Configuration

Student Chapter

Learning outcome: By the end of Day 34, you should be able to explain, implement, test and debug the core outcomes of Duck DNS & SSL Configuration.

Day 34 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Duck DNS & SSL Configuration.
- Demonstrate the end-to-end request or feature flow for Duck DNS & SSL Configuration.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. What is DNS

What is DNS moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain What is DNS in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. What is Duck DNS

What is Duck DNS moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain What is Duck DNS in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Create Free Domain

Create Free Domain is a core part of Duck DNS & SSL Configuration. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Create Free Domain in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Domain Mapping to EC2

Domain Mapping to EC2 moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Domain Mapping to EC2 in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Static IP Discussion

Static IP Discussion belongs to the presentation layer. Its purpose is to turn application data and user actions into a clear, accessible interface.

Keep business decisions in Python and presentation decisions in templates or React components. Demonstrate how variables cross the backend–frontend boundary and how missing or empty data should be rendered.

Teach reuse early through template inheritance, includes or reusable components. Ask yourself to inspect generated HTML and network requests rather than treating the interface as magic.

Check your understanding: Explain Static IP Discussion in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Install SSL Certificate

Install SSL Certificate is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Install SSL Certificate in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. HTTPS Configuration

HTTPS Configuration is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain HTTPS Configuration in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Nginx SSL Setup

Nginx SSL Setup is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Nginx SSL Setup in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Duck DNS & SSL Configuration flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 34 reference example

```
server {  
    listen 80;  
    server_name your-subdomain.duckdns.org;  
    location / { proxy_pass http://127.0.0.1:8000; }  
}  
# Then obtain and configure a TLS certificate.
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Duck DNS & SSL Configuration, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

- 199.Reproduce the failure consistently.
- 200.Read the status code and traceback from the top-level error to the relevant application line.
- 201.Print or log only safe diagnostic values.
- 202.Verify the active route, method and request payload.
- 203.Test the database or external service independently.
- 204.Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 34 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 34 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 34 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Duck DNS & SSL Configuration.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

DAY 35

Final Production Testing

Student Chapter

Learning outcome: By the end of Day 35, you should be able to explain, implement, test and debug the core outcomes of Final Production Testing.

Day 35 Learning Roadmap

Learning Objectives

- Explain the purpose and vocabulary of Final Production Testing.
- Demonstrate the end-to-end request or feature flow for Final Production Testing.
- Implement a working example using safe Flask practices.
- Test the successful path and important failure paths.
- Communicate the design using a diagram, code walkthrough and verbal explanation.

Suggested Learning Flow

Block	Focus	Evidence
Preview	Recall prior learning and understand the user problem.	Write the expected behaviour in your own words.
Learn	Study vocabulary, architecture, rules and edge cases.	Draw the flow and answer concept checks.
Guided build	Type and run the minimal working implementation.	Inspect application output, network requests and database changes.
Practice	Rebuild with checkpoints and small changes.	Create working intermediate commits.
Independent lab	Extend the feature and test negative cases.	Complete acceptance checklist and capture evidence.
Review	Self-test, debugging and summary.	Explain the code without reading it.

Prerequisite Recall

- What request or user action starts today's feature?
- Which data enters the application and where does it come from?
- What must be validated before the feature proceeds?
- Which previous route, template, table or service will be reused?
- What should the user see when the operation succeeds or fails?

Conceptual Deep Dive

1. Test Authentication APIs

Test Authentication APIs is security-sensitive. Learners must understand the threat being addressed before memorising the implementation.

Clarify trust boundaries: browser input is untrusted, identifiers in URLs can be manipulated, passwords must never be stored in plain text, and authentication does not automatically guarantee authorization.

Demonstrate expiry, invalid values and replay attempts. Ask yourself to explain what information is stored, where it is stored, how it is verified and what happens after logout or expiration.

Check your understanding: Explain Test Authentication APIs in your own words using the current project. Then write one possible failure or security risk and how you would test it.

2. Test Product APIs

Test Product APIs is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Test Product APIs in your own words using the current project. Then write one possible failure or security risk and how you would test it.

3. Test Cart APIs

Test Cart APIs is part of the HTTP interface between a client and Flask. Connect the URL and method to a specific application action and response contract.

Show the request in the browser Network panel or Postman. Identify query parameters, form values, JSON body, headers and cookies, and explain which Flask object exposes each value.

A route should validate input, enforce authentication and authorization where required, call application or persistence logic, and return an appropriate HTML, redirect or JSON response with a meaningful status code.

Check your understanding: Explain Test Cart APIs in your own words using the current project. Then write one possible failure or security risk and how you would test it.

4. Test Razorpay Payment Flow

Test Razorpay Payment Flow describes how responsibilities and data move between parts of the application. You should draw the participating components first, then trace one user action from beginning to end.

Learners should identify the starting event, the request payload, the validation point, the business rule, the persistence action and the final response. This prevents them from seeing the application as disconnected files.

A reliable explanation always includes both the successful path and at least one failure path. Ask what happens when input is missing, a record does not exist, authorization fails or an external service is unavailable.

Check your understanding: Explain Test Razorpay Payment Flow in your own words using the current project. Then write one possible failure or security risk and how you would test it.

5. Test Order Generation

Test Order Generation is a transactional feature. The browser can initiate a payment, but the server must remain the authority for amount, order identity and final verification.

Create the order on the server, send only necessary checkout data to the client, verify the payment signature on the server and update the order atomically. Never trust a client-only success callback.

Demonstrate duplicate callbacks, failed payments and network interruption. Learners should understand idempotency and why payment records need auditable status transitions.

Check your understanding: Explain Test Order Generation in your own words using the current project. Then write one possible failure or security risk and how you would test it.

6. Test Invoice Download

Test Invoice Download involves data that is larger or more complex than ordinary text fields. The implementation must control filenames, formats, size, storage location and access permissions.

Never trust the original filename or file extension alone. Generate safe storage names, validate allowed types and size, keep metadata separate from content, and verify record ownership before serving or deleting a file.

Discuss whether the file belongs in local storage, object storage or the database. For classroom work local storage is simple; production design must consider backups, scaling and privacy.

Check your understanding: Explain Test Invoice Download in your own words using the current project. Then write one possible failure or security risk and how you would test it.

7. Error Logging

Error Logging moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Error Logging in your own words using the current project. Then write one possible failure or security risk and how you would test it.

8. Production Checklist

Production Checklist moves the application from a developer laptop to a managed runtime. You should separate application code, process management, networking and configuration.

Explain the production request path: public DNS resolves to a server, Nginx accepts the connection, proxies to Gunicorn, Gunicorn invokes Flask, and logs record application or infrastructure failures.

Secrets must be supplied through environment variables or a secret manager. Production servers should not run Flask debug mode, and deployment should include health checks, rollback planning and verification.

Check your understanding: Explain Production Checklist in your own words using the current project. Then write one possible failure or security risk and how you would test it.

9. Project Demonstration

Project Demonstration is a core part of Final Production Testing. Begin with the user or business problem it solves, then introduce the vocabulary and implementation only after learners can describe the expected behaviour.

Use a minimal example first, inspect the result, then add validation and edge cases. Learners should predict the output before execution and explain each line after it runs.

Connect this concept to earlier lessons and the current project. Require learners to state inputs, processing, output and possible failure conditions in their own words.

Check your understanding: Explain Project Demonstration in your own words using the current project. Then write one possible failure or security risk and how you would test it.

Architecture and Request Flow

Whiteboard flow

```

Client / Browser / Postman
    |
    v
Flask URL rule and HTTP method
    |
    v
Input extraction and validation
    |
    +--> Authentication / authorization
    +--> Database / file / email / payment service
    |
    v
HTML, redirect, file or JSON response
  
```

Use the flow as a reusable diagnostic model. When a feature fails, locate the layer first: route match, request parsing, validation, session state, business rule, database or external service, response construction, or frontend rendering. This is faster than changing multiple lines at random.

How to Think Through the Flow

- “The user begins the Final Production Testing flow by performing a specific action in the client.”
- “Flask matches both the URL and the HTTP method before our function runs.”
- “We treat all client data as untrusted and validate it before changing state.”
- “The server, not the browser, is responsible for enforcing ownership and business rules.”
- “Our response must communicate both machine-level status and user-level meaning.”

Guided Demonstration

Day 35 reference example

```
curl -i https://your-domain.example/api/products
curl -i -X POST https://your-domain.example/api/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"test@example.com","password":"Example123!"}'
```

Line-by-Line Walkthrough Questions

- Which imports are framework features and which are Python standard-library or third-party features?
- What URL and method cause this code to execute?
- Where does input enter the function?
- What validation is performed and what is still missing?
- Which state is read or changed?
- What exact response is returned?
- How would the code behave with an empty value, invalid identifier, expired session or service failure?

Extension Demonstration

After the minimal path works, extend the demonstration with one additional validation rule, one explicit failure response and one logging statement. For Final Production Testing, ask learners to predict the outcome before each run. Use the browser Network panel, Flask console, database client or server logs to verify the prediction.

Common Mistakes and Debugging

Mistake	How to fix it
Route or method mismatch	The URL exists but the HTTP method is not registered, causing 404 or 405 behaviour.
Input source confusion	The code reads <code>request.form</code> while the client sends JSON, or reads query parameters for a POST body.
Missing validation	Empty, malformed or unauthorized input reaches database or service code.
Unsafe database handling	String-formatted SQL creates correctness and security problems.
Authentication without authorization	A logged-in user can access another user's record by changing an identifier.
Debugging by random edits	The student changes many layers without first locating where the expected flow stops.
Secrets in source code	Credentials or secret keys are committed instead of loaded from environment variables.
Only testing the happy path	The feature appears complete until invalid input or a missing dependency occurs.

Debugging Drill

- 205.Reproduce the failure consistently.
- 206.Read the status code and traceback from the top-level error to the relevant application line.
- 207.Print or log only safe diagnostic values.
- 208.Verify the active route, method and request payload.
- 209.Test the database or external service independently.
- 210.Apply one change, rerun the same test and record the result.

Student Lab and Acceptance Criteria

Lab Brief

Add the Day 35 milestone to the E-Commerce application. Demonstrate the feature, record test evidence and update deployment or API documentation where applicable.

Minimum Acceptance Criteria

- The Day 35 feature works for the intended user flow.
- Input is validated before state is changed.
- Database queries use placeholders where applicable.
- Protected data is scoped to the authenticated user or administrator.
- At least one invalid-input case and one missing-record or service-failure case are tested.
- The interface or API returns a clear success and failure response.
- The student can explain the complete request lifecycle.
- The stable milestone is committed with a meaningful Git message.

Stretch Tasks

- Add a reusable helper or decorator without hiding the request flow from beginners.
- Improve accessibility or empty-state presentation.
- Add logging that helps diagnose failures without exposing secrets.
- Create a Postman request or manual test case for the feature.
- Refactor repeated code only after tests confirm behaviour is unchanged.

Viva, Review and Assignment

Question	Expected answer
What starts this feature?	A specific client action mapped to a URL and HTTP method.
Where is input read?	From route variables, query parameters, form data, JSON, files, headers or cookies depending on the contract.
Why must input be validated on the server?	Client-side controls can be bypassed and server rules protect data integrity.
What is the difference between authentication and authorization?	Authentication identifies the user; authorization decides whether that user may perform the action.
Why use parameterized SQL?	It separates query structure from values and prevents SQL injection.
What should happen when a record is missing?	Return or render an appropriate not-found response without exposing internals.
How do you test a failure path?	Create controlled invalid input or dependency failure and verify status, message and unchanged state.
What belongs in environment variables?	Secrets and environment-specific settings such as keys, credentials and service URLs.
Why is debug mode unsafe in production?	It can expose sensitive details and interactive debugging capabilities.
What evidence proves the feature is complete?	Passing acceptance criteria, negative tests, clear explanation and a stable repository milestone.

Day-End Assignment

Independently rebuild or extend the Day 35 feature. Submit source code, a short README, two screenshots or API responses, one negative-test result and a paragraph explaining the request lifecycle. The assignment topic is Final Production Testing.

Student Submission Rubric

Area	Marks	Evidence
Conceptual explanation	20	Correct vocabulary and complete flow.
Implementation	30	Working feature with readable code.
Validation and security	20	Server validation, safe queries and correct access control.
Testing and debugging	20	Positive and negative evidence; systematic diagnosis.
Documentation and Git	10	README, meaningful commit and clean repository.

Appendix A — Production Readiness Checklist

- Debug mode is disabled.
- Secret keys and credentials are not committed.
- HTTPS is enabled.
- Cookies use appropriate Secure, HttpOnly and SameSite settings.
- All state-changing routes validate input and authorization.
- Database accounts use least privilege.
- Uploads have type, size, naming and ownership controls.
- Errors are logged; users receive safe messages.
- Dependencies are pinned and reviewed.
- Backups, monitoring, health checks and rollback procedures are documented.

Appendix B — Final Project Demonstration Script

211. Introduce the user problem and architecture.
212. Demonstrate registration and authentication.
213. Show role or ownership controls.
214. Create, read, update and delete core records.
215. Demonstrate file or image handling.
216. Show API requests and status codes.
217. Complete the cart, payment, order and invoice flow.
218. Open deployment configuration and explain Nginx, Gunicorn and systemd.
219. Trigger one controlled error and show logs and safe user response.
220. Conclude with limitations and next improvements.

Appendix C — Student Interview Practice Bank

No.	Question
1	Explain request lifecycle in a Flask application and give one failure case the server must handle.
2	Explain routing in a Flask application and give one failure case the server must handle.
3	Explain Jinja2 in a Flask application and give one failure case the server must handle.
4	Explain sessions in a Flask application and give one failure case the server must handle.
5	Explain cookies in a Flask application and give one failure case the server must handle.
6	Explain SQL safety in a Flask application and give one failure case the server must handle.
7	Explain authentication in a Flask application and give one failure case the server must handle.
8	Explain authorization in a Flask application and give one failure case the server must handle.
9	Explain file security in a Flask application and give one failure case the server must handle.
10	Explain REST APIs in a Flask application and give one failure case the server must handle.
11	Explain payment verification in a Flask application and give one failure case the server must handle.
12	Explain Gunicorn in a Flask application and give one failure case the server must handle.
13	Explain Nginx in a Flask application and give one failure case the server must handle.
14	Explain DNS and TLS in a Flask application and give one failure case the server must handle.
15	Explain React integration in a Flask application and give one failure case the server must handle.
16	Explain request lifecycle in a Flask application and give one failure case the server must handle.
17	Explain routing in a Flask application and give one failure case the server must handle.
18	Explain Jinja2 in a Flask application and give one failure case the server must handle.
19	Explain sessions in a Flask application and give one failure case the server must handle.
20	Explain cookies in a Flask application and give one failure case the server must handle.
21	Explain SQL safety in a Flask application and give one failure case the server must handle.
22	Explain authentication in a Flask application and give one failure case the server must handle.
23	Explain authorization in a Flask application and give one failure case the server must handle.
24	Explain file security in a Flask application and give one failure case the server must handle.
25	Explain REST APIs in a Flask application and give one failure case the server must handle.
26	Explain payment verification in a Flask application and give one failure case the server must handle.
27	Explain Gunicorn in a Flask application and give one failure case the server must handle.
28	Explain Nginx in a Flask application and give one failure case the server must handle.
29	Explain DNS and TLS in a Flask application and give one failure case the server must handle.
30	Explain React integration in a Flask application and give one failure case the server must handle.
31	Explain request lifecycle in a Flask application and give one failure case the server must handle.
32	Explain routing in a Flask application and give one failure case the server must handle.
33	Explain Jinja2 in a Flask application and give one failure case the server must handle.
34	Explain sessions in a Flask application and give one failure case the server must handle.
35	Explain cookies in a Flask application and give one failure case the server must handle.
36	Explain SQL safety in a Flask application and give one failure case the server must handle.
37	Explain authentication in a Flask application and give one failure case the server must handle.

No.	Question
38	Explain authorization in a Flask application and give one failure case the server must handle.
39	Explain file security in a Flask application and give one failure case the server must handle.
40	Explain REST APIs in a Flask application and give one failure case the server must handle.
41	Explain payment verification in a Flask application and give one failure case the server must handle.
42	Explain Gunicorn in a Flask application and give one failure case the server must handle.
43	Explain Nginx in a Flask application and give one failure case the server must handle.
44	Explain DNS and TLS in a Flask application and give one failure case the server must handle.
45	Explain React integration in a Flask application and give one failure case the server must handle.
46	Explain request lifecycle in a Flask application and give one failure case the server must handle.
47	Explain routing in a Flask application and give one failure case the server must handle.
48	Explain Jinja2 in a Flask application and give one failure case the server must handle.
49	Explain sessions in a Flask application and give one failure case the server must handle.
50	Explain cookies in a Flask application and give one failure case the server must handle.
51	Explain SQL safety in a Flask application and give one failure case the server must handle.
52	Explain authentication in a Flask application and give one failure case the server must handle.
53	Explain authorization in a Flask application and give one failure case the server must handle.
54	Explain file security in a Flask application and give one failure case the server must handle.
55	Explain REST APIs in a Flask application and give one failure case the server must handle.
56	Explain payment verification in a Flask application and give one failure case the server must handle.
57	Explain Gunicorn in a Flask application and give one failure case the server must handle.
58	Explain Nginx in a Flask application and give one failure case the server must handle.
59	Explain DNS and TLS in a Flask application and give one failure case the server must handle.
60	Explain React integration in a Flask application and give one failure case the server must handle.